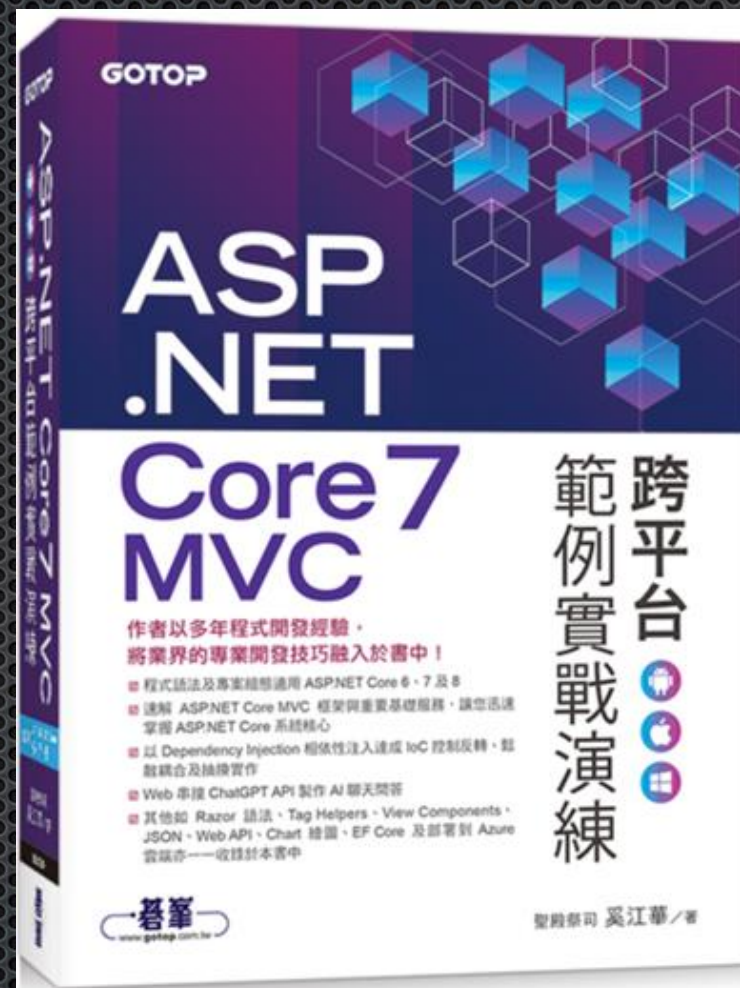


第13章

以Dependency Injection相依性注入 達成IoC控制反轉



作者：聖殿祭司。奚江華

大綱

13-1 DI 相依性注入概觀

13-2 ASP.NET Core 內建的 DI 相依性注入

13-3 在 Controller 及 Action 使用相依性注入

13-4 在Views中使用相依性注入

13-5 相依性注入服務之生命週期

13-6 註冊服務之方式

DI 相依性注入概觀

13-1

何謂 Dependency Injection ?

- DI中文稱相依性注入或依賴注入
- 先就字義拆解，所謂的 Dependency 翻譯成依賴、相依性，指的是一個東西被另一個東西需要
- 例如空氣與人的關係，人需要呼吸空氣，因而對空氣產生依賴性，那麼空氣就是 Dependency 相依性物件

- Injection中文是注入，如果把 Dependency 依賴的東西傳遞給需求者，這就是注入，而不是由需求者主動建立或搜尋
- 例如人們到百貨公司逛街購物，需要新鮮的空氣，百貨公司空調就會自動注入提供給你，不需你費力取得，這就是 Dependency Injection 相依性注入的意思

直接相依特定類別實作之困境

- 例如網站串接到台北富邦網路銀行服務 FubonBankService，存取包括餘額查詢、轉帳等服務

```
public class FubonBankService : IBankService
{
    public string BankId { get; private set; }

    public string BankName { get; private set; }

    public FubonBankService()
    {
        BankId = "012";
        BankName = "台北富邦銀行";
    }

    public decimal AccountBalance(string depositorId)
    {
        decimal balance = 1000000;
        if (depositorId == "18072")
        {
            balance = 5000000;
        }

        return balance;
    }
}
```

Services/FubonBankService.cs

Services/FubonBankService.cs

```
public bool Deposit(decimal dollars)
{
    //...
    return true;
}

public bool Withdraw(decimal dollars)
{
    //...
    return true;
}
}
```

傳統程式調用 FubonBankService 類別實例

- 先初始化 FubonBankService 類別實例，再呼叫 AccountBalance 方法查詢銀行餘額

```
FubonBankService _bankService = new FubonBankService();  
_bankService.AccountBalance(10354)
```

直接相依會產生什麼問題？

- 直接初始化特定類別加以調用，會造成程式間非常強烈的緊密耦合
- 在軟體設計上，緊密耦合是不好，因為牽一髮動全身
- 一旦要網銀要換成另一家玉山銀行的 **ESunBankService** 服務，就要對整個專案程式全面修改和檢測

```
EsunBankService _bankService = new ESunkService();
```

問題還沒完~

- 更慘的是，ESunBankService 查詢銀行餘額的方法名稱是GetBalance，與之前FubonBankService類別的 AccountBalance 不同

```
_bankService.GetBalance(10354)
```

- 但這樣就完了嗎？不，所有網銀程式都必須用人力全面重新檢查及測試過，找出哪些地方不相容或有問題
- 甚至兩個網銀服務的方法名稱、參數與回傳值完全不同都十分可能，想到這，換一個服務就必須將整個專案程式做檢測與大改寫

小結相依特定實作的缺點

- 程式直接相依特定類別實作，會造成強列耦合，一旦需更換類別實作，必須全面修改專案中相依特定類別所有程式
- 如果相依的實作它自己也有相依其他物件，例如 Configuration 或 Log，那麼這些組態就的設定就會散佈在專案的各個角落，造成組態上的凌亂
- 且由於未實作 interface 介面，造成單元測試要 Mocking 或 Stubbing該類別實作會有困難，變得難以進行單元測試

改用DI相依性注入的救贖

- ① 相依性注入會先定義 interface 介面，類別再繼承此介面並實作。讓程式相依介面，而不直接相依特定類別實作，進而抽象化類別實作，就不會造成緊密耦合，變成鬆散耦合
- ② interface 介面會定義屬性與方法，強制繼承的類別必須實作，因而有共通標準 API，即使更換服務也必須實作相同的 API 介面，相同的參數與回傳結果，讓程式不受更換服務影響
- ③ 由於繼承並實作 interface 介面，使得單元測試 Mocking 或 Stubbing變得非常容易

ASP.NET Core 內建的 DI 相依性注入

13-2

ASP.NET Core 內建的 DI

- ASP.NET Core內建支援Dependency Injection相依性注入
- 它是用來將類別與其相依性達成 IoC 控制反轉 (Inversion of Control) 之目的，讓程式間變成鬆散耦合
- ASP.NET Core 內建原生的 DI Container 容器 (IoC Container 或 Service Container)，用來支援建構函式注入 (Constructor Injection)

```
var builder = WebApplication.CreateBuilder(args);
```

```
// DI Container區段
```

```
var app = builder.Build();
```

Program.cs

DI相關名詞說明

- 控制反轉有兩個層面意義，一是相依性物件的獲得被反轉了，二是控制權被反轉
- 實現控制反轉方式有兩種，一是相依性注入，另一是相依性查找（lookup），前者是被動取得相依性注入的物件，後者則是透過物件查找
- DI Container、IoC Container 和 Service Container 三者係指相同東西
- DI Container 是相依性注入的框架實作，管理相依性物件的建立與生命週期，同時亦負責將相依性物件注入到服務或類別中

相依性注入優點

相依性注入好處

- ① 抽象化特定實作
- ② 鬆散耦合
- ③ 易於單元測試

ASP.NET Core 的 DI 實作過程

- 使用 interface 來抽象化 Dependency 相依性實作
- 透過 Service Container 註冊 Dependency 「相依性」和「實作」間的關係。ASP.NET Core 內建的 Service Container – `IServiceProvider`，而相依性服務註冊是在 **Program.cs 的 DI Container**
- 使用相依性服務時，是在類別建構函式中注入服務實例。而 DI 框架負責：**1. 建立相依服務的 instance 實例**，**2. 注入的相依性物件不再用到時的 disposing**

相依性注入的最佳設計原則

相依性注入的最佳設計原則

- 設計DI服務時應避免狀態 / 靜態類別與方法
- 避免建立Global全域狀態，應使用Singleton服務的設計
- 避免在 Services 中直接實例化相依性類別，因為會導致程式和特定實作緊密耦合
- Services 服務要維持小巧、結構良好、易於測試

在 Controller 及 Action 使用相依性注入

13-3

ASP.NET中支援DI注入方式

- ① 在Controller建構函式注入
- ② 在Action方法注入
- ③ 在View中注入

在 Controller 建構函式 使用相依性注入

13-3-1

在 Controller 建構函式使用相依性注入

```
public class BankController : Controller
```

```
{
```

```
    private readonly IBankService _bankService;
```

1.宣告全域變數

```
    public BankController(IBankService bankService)
```

```
{
```

```
        _bankService = bankService;
```

2.注入服務實例

```
}
```

3.將區域變數指派給全域變數，供所有Action存取

```
    . . .
```

```
}
```

Controllers/BankController.cs

ASP.NET Core 使用相依性注入之步驟

- ① 建立interface介面，宣告屬性與方法
- ② 建立Service服務類別，繼承interface介面並實作
- ③ 在 Program.cs的DI Container註冊Service相依性
- ④ 在 ASP.NET Core 的 Controller、Action 或 View 中注入 Service 相依性物件，然後使用

範例 13-1 網銀服務之 DI 相依性注入建立與調用

Balance - CoreMvc3_Dependen x +

localhost:5001/Bank/Balance/

DI Home Privacy IBankService相依性注入 ▾

Balance帳戶餘額查詢

網路銀行Dependency Injection相依性注入服務

- 銀行代碼 : 012
- 銀行名稱 : 台北富邦銀行
- 存款餘額 : NT\$5,000,000.00

Balance - CoreMvc3_Dependen x +

localhost:5001/Bank/Balance/

DI Home Privacy IBankService相依性注入 ▾

Balance帳戶餘額查詢

網路銀行Dependency Injection相依性注入服務

- 銀行代碼 : 808
- 銀行名稱 : 玉山銀行
- 存款餘額 : NT\$1,500,000.00

在 Action 使用相依性注入

13-3-2

在 Action 使用相依性注入

- 在 Action 的參數位置用[FromServices]宣告服務注入

```
public IActionResult InjectAction([FromServices] IBankService myBankService)
{
    return View(myBankService);
}
```

在Action中注入

在Views中使用相依性注入

13-4

在Views中使用DI時機

- 技術上，View 也能直接注入相依性服務。但為了維持 SoC 關注點分離原則，相依性物件或資料最好還是由 Controller 傳遞給 View
- 何時適合在 View 直接注入相依性？就是要注入的服務、資料純粹屬於 View 前端，無涉 Controller、Model 和中後端邏輯情況下

View直接做相依性注入四類應用

- ① Service 服務注入
- ② 填入查閱資料
- ③ Configuration 組態注入
- ④ 覆寫服務

將 Service 服務相依性注入 View

13-4-1

將 Service 服務相依性注入 View

- 以技術角度出發，展示如何將網銀服務直接注入 View 中，方式是在 View 用 `@inject` 指示詞宣告相依性注入

```
@using AspCoreOverview.Interfaces
```

```
@inject IBankService bankService
```

```
...
```

用 `@inject` 宣告相依性注入

```
<ul>
```

```
<li>銀行代碼 : @bankService.BankId</li>
```

```
<li>銀行名稱 : @bankService.BankName</li>
```

```
<li>存款餘額 : @bankService.AccountBalance("10872")</li>
```

```
</ul>
```

`IBankService` 相依性物件實例

避免誤用

- 前面只是技術性示範，因網銀服務算是中後端邏輯與服務，並不屬於純粹的前端範疇，以 SoC 的角度，透過 Controller 注入相依性物件，再以 model 傳遞資料給 View 是較理想方式
- 較正確的使用法是，該服務屬於前端服務，專為前端網頁產生 UI 介面資料，而不涉及中後端服務或商業邏輯，此時在 View 中直接注入服務就有其正當性

範例 13-2 將 IZipCodeService 服務注入 View , 用以查詢郵遞區號

InjectZipcodeService - CoreMv x +

localhost:5001/ViewInjection/InjectZipcodeService/

DI Home Privacy IBankService相依性注入 Contrller相依性注入 View相依性注入

Inject ZipcodeService

將ZipcodeService注入到View

- 基隆市,七堵區 Zip code : 206
- 台北市,信義區 Zip code : 110
- 新北市,板橋區 Zip code : 220
- 桃園市,八德區 Zip code : 334
- 銀河市,火星區 Zip code : 查無此City
- 桃園市,火星區 Zip code : 查無此District

透過 Service 注入 View 並將資料填入 UI 介面

13-4-2

透過注入的 Service 服務將資料填入 UI 介面

- 本節要將相依性注入與 UI 介面資料的結合，透過注入的 Service 服務將資料填入 UI 介面
- 例如網頁中常會用到縣市名稱下拉式選單，透過注入的「City 服務」提供 View 的下拉式選單 UI所需縣市名稱資料（無關後端邏輯與服務）

範例 13-3

透過 Service 注入 City 縣市資料到下拉式選單 UI



- CoreMvc3_DependencyInject x +

localhost:5001/Di/InjectCityService/

用 SelectListItem 集合資料顯示 City

==請選擇縣市== ▾ Submit

將 List 轉換成 SelectListItem 集合, 顯示 City

==請選擇縣市== ▾ Submit

用 foreach 逐筆讀取 Cities 資料

- 01, 基隆市
- 02, 臺北市
- 03, 新北市
- 04, 桃園市

將 Configuration 組態注入到 View

13-4-3

appsettings.json 中組態

```
{
  "Company": {
    "Website": "https://www.codemagic.com.tw",
    "Branches": {
      "Taipei": {
        "Name": "台北總公司",
        "Tel": "0800-000-123"
      },
      "Taichung": {
        "Name": "台中分公司",
        "Tel": "0800-111-456"
      },
      "Kaohsiung": {
        "Name": "高雄分公司",
        "Tel": "0800-222-789"
      }
    }
  }
}
```

在View中注入IConfiguration 相依性物件

```
@using Microsoft.Extensions.Configuration  
@inject IConfiguration configuration
```

注入IConfiguration相依性物件

```
@{  
    string Website = configuration["Company:Website"];  
    string Taipei_Name = configuration["Company:Branches:Taipei:Name"];  
    string Taipei_Tel = configuration["Company:Branches:Taipei:Tel"];  
    string Taichung_Name = configuration["Company:Branches:Taichung:Name"];  
    string Kaohsiung_Name = configuration["Company:Branches:Kaohsiung:Name"];  
}
```

顯示組態值

組態階層用:分隔

```
<ul>  
    <li>公司網站 : @Website</li>  
    <li>@Taipei_Name , 電話 : @Taipei_Tel</li>  
    <li>@Taichung_Name , 電話 : @Taichung_Tel</li>  
    <li>@Kaohsiung_Name , 電話 : @Kaohsiung_Tel</li>  
</ul>
```

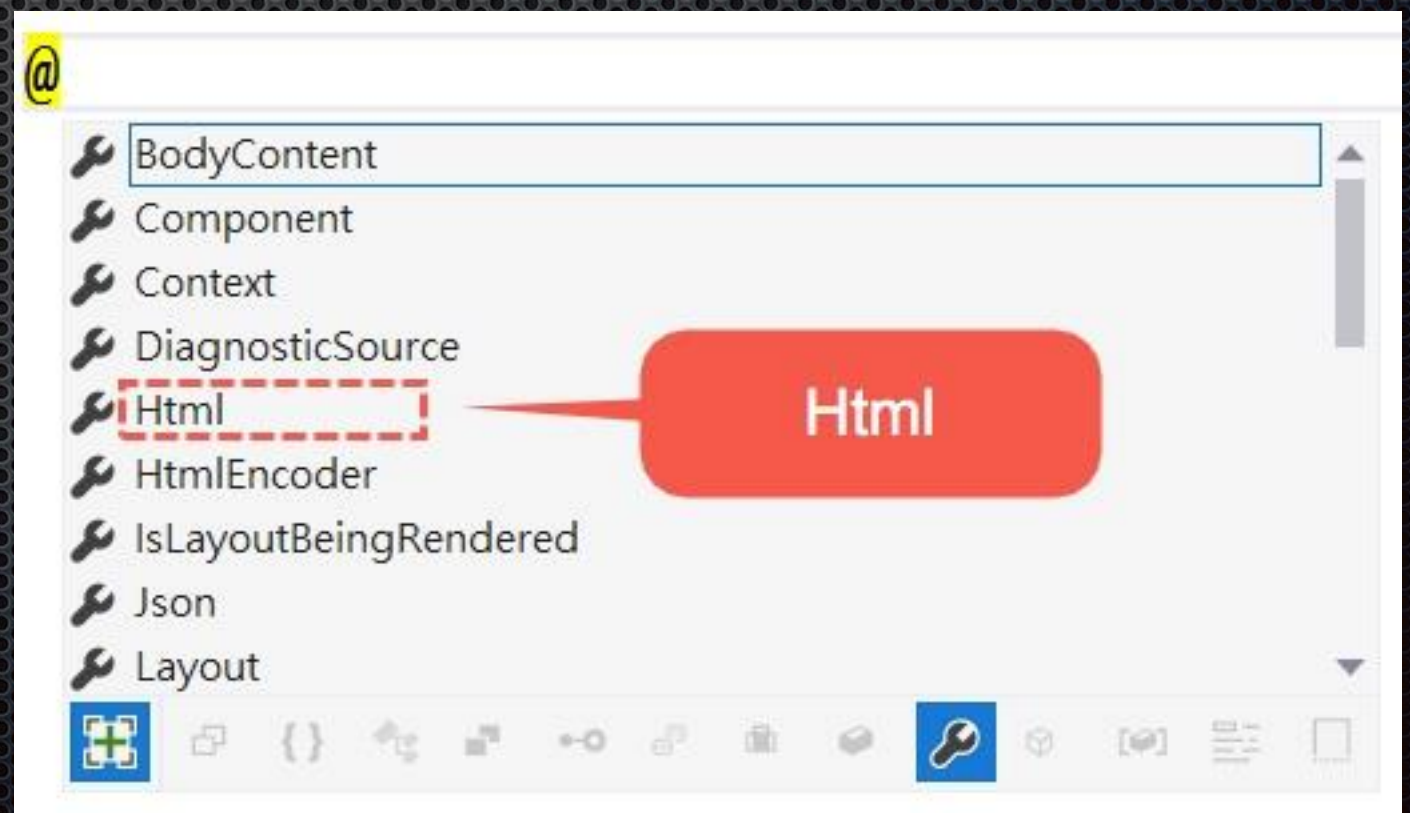
Views/ViewInjection/InjectConfiguration.cshtml

覆寫服務

13-4-1

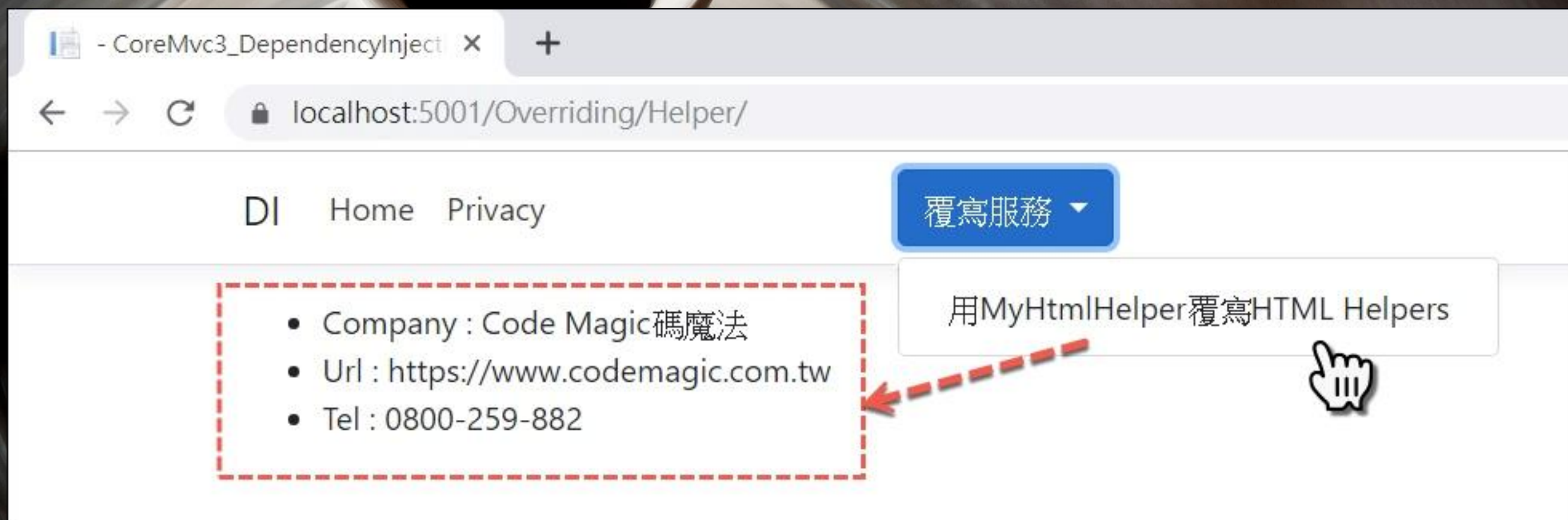
以注入的Service覆寫系統服務

- 例如在 View 中輸入@關鍵字，IntelliSense 會提示可使用的關鍵字，其中一個是Html，它是系統的 HTML Helpers
- 若你建立一個服務，在注入 View 頁面時也取名為 Html，那麼系統預設的 Html Helpers 就會被覆寫掉，變成你自訂的 Html



範例 13-5

用自訂 MyHtmlHelper 覆寫預設的 HTML Helpers



相依性注入服務之生命週期

13-5

相依性注入服務之生命週期

在 DI Container 中註冊 Service 服務，依其使用的方法不同，有三種生命週期：

- Transient (短暫的)
- Scoped (範圍)
- Singleton (單一)

註冊服務的三種Lifetime指令

- **AddTransient**方法 — **Transient** lifetime service
 - are created each time they're requested from the service container
- **AddScoped**方法 — **Scoped** lifetime service
 - are created once per client request (connection for Web)
- **AddSingleton**方法 — **Singleton** lifetime service
 - The first time they're requested

Transient (短暫的)

- 使用 AddTransient 方法註冊的服務，就是 Transient 生命週期。其特性是每次向 Service Container 請求服務時，此服務實例就會被建立（意謂服務可能建立多次）
- Request 結束後就由 DI Container 負責 Dispose，此存留期最適合用於輕量級 / 無狀態服務
- By User

Scoped (範圍)

- 使用 AddScoped 方法註冊的服務，就是 Scoped 生命週期。其特性是在每個用戶端請求(per client request, connection)，無論向Service Container請求幾次服務，服務僅建立一次
- EF Core註冊AddDbContext方法內部實作就是使用 Scoped
- Request結束後就由DI Container負責Dispose
- By User

Singleton (單一)

- 使用 AddSingleton 方法註冊的服務，就是 Singleton 生命週期。其特性是此服務第一次被請求（或當 Startup.ConfigureServices 方法執行時），就會建立單一實例
- 此服務實例就會一直存留，讓後續所有用戶端使用相同的服務實例
- 當應用程式Shutdown後，Service Provider會被Disposed，接著Singleton服務跟著被Disposed
- By Website

不同長度生命週期Services彼此以DI注入調用的注意事項

- Do not resolve a scoped service from a singleton. 避免處理後續請求有狀態不正常狀態發生 (**避免長呼叫短**)
- Resolve a singleton service from a scoped or transient service (**短叫長是OK的**)
- Resolve a scoped service from another scoped or transient service (**短叫長是OK的**)

註冊服務之方式

13-6

DI註冊服務幾個未注意問題

- 註冊服務的方法，是否清一色都是 `builder.Services.Add{生命週期}<介面,實作>()` 形式，有沒有其他不同的語法？
- 如果都是 `builder.Services.Add{生命週期}<介面,實作>()` 形式，那萬一要傳遞參數給實作類別，該怎麼辦？
- DI Container 會自動 Dispose 掉服務實例（當服務不再需要時），是否絕對沒有例外？

服務在 DI Container 註冊方式

註冊方式	自動物件 Disposal	支援 多重實作	傳遞 參數
Add{生命週期} <{介面},{實作}>() 例如： services.AddTransient<IBankService, FubonBankService>();	Yes	Yes	No
Add{生命週期} <{介面}>(sp => new {實作}) 例如： services.AddTransient<IService>(sp => new MyService ()); services.AddTransient<IService>(sp => new MyService (參數));	Yes	Yes	Yes
Add{生命週期} <{實作}>() 例如： services.AddScoped<MyHtmlHelper>();	Yes	No	No
Add{生命週期} <{介面}>(new {實作}) 例如： services.AddSingleton<IService>(new MyService()); services.AddSingleton<IService>(new MyService (參數));	No	Yes	Yes
Add{生命週期}(new {實作}) 例如： services.AddScoped(new MyService ()); services.AddScoped(new MyService (參數));	No	No	Yes

The End

奚江華個人經歷簡介

- 著有ASP.NET 2.0、3.5、4.5、4.6等熱門暢銷書籍，以及**ASP.NET MVC 5.x範例完美演繹、ASP.NET Core 3.x、ASP.NET Core 7 MVC跨平台範例實戰演練**
- 歷任台灣微軟MSDN、TechEd、TechDay研討會講師
- 曾數屆當選微軟MVP
- 國內各大上市公司及大專院校之培訓講師
- 曾發表文章刊於ITHOME電腦週刊

碼魔法FB

- CodeMagic碼魔法軟體學院Facebook

www.facebook.com/codemagictw